Lab 05: The Earth Twin Awakens

12062 Algorithmic Robotics | Week 5 | 2 Hours

Mission Briefing

Three weeks ago, Kevin from Accounting launched himself toward Mars in *Pineapple One* with more confidence than preparation. He is now somewhere on the Martian surface, alive (we think), and almost certainly filing complaints about the signal-to-noise ratio. Tim Apple has authorised a full rescue operation. Here is the plan:

- **Succulence** -- our ground rover -- will navigate the Martian terrain, build a map on the fly, and extract Kevin once his location is confirmed.
- **The FruitFly drone swarm** (fruitfly_1 through fruitfly_4) -- will fan out across the surface, survey a wide area, and pinpoint Kevin's exact coordinates.

The problem: Mars is approximately 225 million kilometres away, and nobody wants to brick the actual hardware on the first run. This is where you come in.

Real space agencies do exactly what you are about to do. NASA's Jet Propulsion Laboratory never sends a command to a Mars rover without first testing it on a full-scale physical copy here on Earth. Perseverance's Earth-based copy, named OPTIMISM, is formally called a **Vehicle System Test Bed (VSTB)**. It rolls around a simulated Martian landscape at JPL before any command is uplinked to the real rover. Engineers call these physical copies **Earth twins**. Your setup mirrors this concept exactly:

Mars (the real thing)	Earth Twin / VSTB	Role
Succulence rover	TurtleBot (Succulence VSTB)	Ground mapping + extraction
FruitFly drones (x4)	Crazyflie drones (x4 VSTBs)	Aerial search swarm
Martian terrain (Unity simulation)	Mars recreation track in the lab	Command validation environment

Over the next few weeks we are going to need to **argmax hard** to find Kevin. Every decision -- every map, every pose estimate, every planned path -- is an optimisation. The mission starts today: bring the Earth twins online, understand the data, and map out the system. Kevin is counting on it.

Task 01: Bring the Earth Twin Online (Physical TurtleBot) 🤖

1.1 Launch the Physical Robot Environment

Navigate to the robotics world repository and bring up the physical robot stack:

```
cd algorithmic-robots-world
docker compose -f compose-physical.yaml pull
xhost +local:root
docker compose -f compose-physical.yaml up
```

Configure your environment first. An `example.env` file is provided in the repository. Rename it to `.env` and update the values to match your assigned robot. Two parameters are critical:

- `ROBOT_ROS_DOMAIN_ID` -- the domain ID of your robot. Check the robot's display panel near the power button, or SSH in and run `echo $ROS_DOMAIN_ID`. If the robot is TBOT4-003, the domain ID is typically 3.
- `ROBOT_ROS_DISCOVERY_SERVER` -- the IP address and port of the robot's discovery server. The IP is shown on the robot's display panel. Port is `11811`. For example: `10.0.0.192:11811`.

> 💡 Hint: To confirm the robot's IP, SSH into it (password: turtlebot4) and run:
> ros2 topic echo /ip or ip a
> and look for the wlan interface address.

1.2 Watch the Earth Twin Move

The TurtleBot already has its controller running onboard -- the robot will be driven around the Mars recreation track during the lab session. **Your job from your local computer is to connect, observe, and explore what is being published.**

Once the stack is up, confirm you can see the robot's topics from your machine:

```
ros2 topic list
```

If you see a populated list, you are connected. If not, double-check your `.env` values and confirm the docker stack is running.

> 💡 Hint: Your local machine is acting exactly like a mission control terminal -- connected to the VSTB on the track, watching its every move. This is the same relationship JPL engineers have with their rover twins.

1.3 Find the Relevant Topics

Run `ros2 topic list` and examine the output carefully. Your goal is to identify topics that will matter for SLAM. Think in two categories:

- **Inputs to SLAM:** what sensor data feeds into the mapping algorithm?
- **Navigation outputs:** how will the system eventually command Succulence to drive to Kevin?

Write these down -- you will need them in Task 03.

1.4 Visualise in RViz

Launch RViz from inside the docker workspace. To add a topic display:

1. Click the **Add** button at the bottom-left of the Displays panel.
2. In the dialog that appears, select the **By topic** tab -- this shows all currently publishing topics.
3. Select the topic you want to visualise and click **OK**.
4. The display will appear in the left-hand Displays panel. Expand it to see its settings -- colour, size, decay time, and so on.

Add displays for the topics you identified in step 1.3. At minimum get the laser scan, odometry, and transform tree visible. Once they are up, **spend time exploring the settings for each display in the left panel** -- change colours, point sizes, or decay times and watch what happens in the 3D view. There is no single right configuration; the goal is to understand what each parameter controls.

> 💡 Hint: The /tf and /tf_static topics describe how all coordinate frames relate to each other -- add a TF display first so you can see the robot's frame structure. If a display shows up red or grey, check that the Fixed Frame (top of the Displays panel) matches a frame that actually exists in /tf.

1.5 Record a ROSbag

ROSbags let you capture live data and replay it later -- extremely useful for offline analysis and debugging without the physical robot running.

Important: record your bag inside the `/workspace` folder inside the docker container. This directory is mapped to a persistent volume on your host machine, so your data survives when the container stops. Navigate there first:

```
cd /workspace
```

Record a bag of your drive session:

```
ros2 bag record -o succulence_run /topic1 /topicN
```

replace topic1, topicN etc with the list of topics you want to record

When done, inspect what you recorded:

```
ros2 bag info succulence_run
```

Replay it and confirm the data streams back correctly:

```
ros2 bag play succulence_run
```

> 💡 Tip: Recording bags throughout this unit is good practice. You will thank yourself in Week 7 when you need data to debug your pose-graph implementation.

Task 02: Connect to Mars (Unity Simulation)

Now bring up the simulated environment -- your Mars. If you have not already cloned the repository, do that first:

```
cd algorithmic-robots-world
docker compose pull
xhost +local:root
docker compose up
```

The same `.env` configuration applies -- rename `example.env` to `.env` and update the values as required before launching. Work through steps 1.2 to 1.5 against the simulation and pay attention to any differences you notice:

- Are the same topics available? Are there any extras, or any missing?
- Does the sensor data look different from the physical TurtleBot?
- How does simulated odometry behave compared to the real robot?

Record a separate bag from the simulation run, labelled clearly so you can tell the two apart later.

Task 03: Study the System Architecture

Download the **succulence_navigation_system_architecture** document from Canvas and have a careful read through it.

This is the full system you will be building over the next few weeks. Scaffolding will be provided -- packages, launch files, and partial implementations -- but you will be responsible for completing the key algorithms and making the critical connections across the system.

Use the topics you identified in Tasks 01 and 02 to orient yourself in the diagram. Where do those topics appear? Which components produce them, and which ones consume them?

Come prepared to discuss what you found with your tutor.

🍍 *Good luck. Kevin has been subsisting on dehydrated mango slices for three weeks and his spreadsheets are getting increasingly erratic. The argmax waits for no one.* 🍍